Code Plagiarism Detection and Generative AI

Peace Samuel

Final Year Project - BSc in Computer Science

Supervisor: Dr. Kieran Herley

Department of Computer Science

University College Cork

April 2025

Abstract

Code plagiarism is the use of another person's code without proper acknowledgment, this is an issue in computer science education, and as a result various techniques have been proposed to handle this issue. With the rise of AI-driven code generation tools, traditional plagiarism detection methods face new challenges, especially in academia. AI-generated code can exhibit structural similarities to human-written code while maintaining syntactical variations. This project explores plagiarism detection in the context of both human-written and AI-generated code, aiming to develop and evaluate an algorithm inspired by the Measure of Software Similarity (MOSS) system.

The MOSS algorithm uses token-based analysis, which generates fingerprints for code samples, and uses various comparator functions to assess similarity. Evaluated against a dataset of plagiarized and non-plagiarized Java code, the algorithm demonstrated high precision and recall in identifying plagiarism.

While standard techniques effectively detect direct plagiarism, AI-generated code introduces a new challenge, as individual code samples may not always be distinguishable from human-written ones. However, this research hypothesizes that AI-generated code exhibits detectable patterns when analyzed at scale, as multiple AI-generated solutions for the same task tend to share underlying similarities. By conducting large-scale similarity comparisons, the study aims to determine whether AI-generated code can be systematically identified. The findings highlight the importance of integrating multiple similarity metrics for more robust detection and lays the groundwork for improving plagiarism detection tools in the era of AI-assisted programming.

Declaration of Originality

In signing this declaration, you are conforming, in writing, that the submitted work is entirely your own original work, except where clearly attributed otherwise, and that it has not been submitted partly or wholly for any other educational award. I hereby declare that:

- this is all my own work, unless clearly indicated otherwise, with full and proper accreditation;
- with respect to my own work: none of it has been submitted at any educational institution contributing in any way to any educational award;
- with respect to another's work: all text, diagrams, code, or ideas, whether verbatim, paraphrased or otherwise modified or adapted, have been duly attributed to the source in a scholarly manner, whether from books, papers, lecture notes or any other student's work, whether published or unpublished, electronically or in print.

Signed: Peace Samuel

Date: 15/04/2025

Contents

1 Introduction

Code plagiarism is the use of another person's code without the proper acknowledgment towards the original work. It is an issue, especially in Computer Science education. The detection of plagiarism is important for supporting the ethical, intellectual and institutional goals of education.

To combat this, a variety of plagiarism detection techniques have been developed, each using different strategies to detect similarities in source code. However, the recent emergence of AI tools capable of generating code has introduced a new challenge for plagiarism detection. These tools can produce highly original-seeming outputs, making traditional plagiarism detection methods less effective.

This project addresses this evolving challenge by developing and evaluating a plagiarism detection algorithm based on an existing method. The chosen approach is inspired by the MOSS (Measure of Software Similarity) algorithm, a widely used technique that relies on tokenization and fingerprinting to detect patterns and similarities in code. The project not only implements this algorithm independently, but also evaluates its effectiveness against both human-written and AI-generated code samples.

2 Objectives

The primary goal of this project is to address the growing challenge of detecting plagiarism in AI-generated code by researching, implementing, and evaluating a plagiarism detection algorithm. This project aims to assess the effectiveness of an existing algorithm when applied to both human-written and AI-generated code.

The study involves the creation of a plagiarism detection algorithm, based on a distinct approach, and then evaluating its performance. The key questions addressed in this study are as follows:

1. How well do existing techniques perform when applied to AI generated code and human-written code?

2. Can AI-specific features or patterns be effectively detected and maybe incorporated into detection algorithms?

By answering these questions the study aims to address the unique challenges posed by generative AI in plagiarism detection.

3 Literature Review

This study began with the research of various existing plagiarism detection techniques, which provide the foundation for the implementation of the program that will be used in this research. These techniques range from simple approaches, such as token-based matching, to more advanced methods like abstract syntax tree analysis. Alongside this the unique challenges posed by AI-generated code were also explored. Additionally, various evaluation metrics were examined to assess the effectiveness of the detection techniques. This literature review will examine each of these topics in detail, providing a comprehensive understanding of the research carried out for this project and the current landscape of plagiarism detection and its relevance in the context of AI-generated code.

3.1 Existing Techniques and Levels of Plagiarism

Source code plagiarism detection has been researched heavily (Novak [2016]), and there are various techniques and tools used. Most existing plagiarism detection techniques work by using a 'searching task', where the degree of similarity between code samples acts as a measure of plagiarism. The files are ranked based on their similarity to each other, effective detection techniques will rank the plagiarized code files as highly similar.

Most existing code plagiarism detection techniques typically work in two consecutive phases: tokenization and comparison.

During the tokenization phase source code files are converted into an intermediate representation. The most common representation for source code is a source code token sequence. Token sequences are generated during the process of breaking down source code into smaller units or 'tokens', these tokens represent meaningful elements of the code, like keywords, variable names, operators, and punctuation. Token sequences have been adopted in a number of techniques such as MOSS. Though it has been argued that they are not comprehensive enough for representing the code files fully, and more advanced representations have been introduced such as: the use of abstract syntax trees that represents source code as its syntactic structure (Fu et al. [2017]) and program dependency graphs which represent how each instruction relies on other instructions (Liu et al. [2006]).

The second phase, the comparison phase measures the similarity between two source code files based on their representations. This phase can be

achieved using various techniques such as structure-based and attribute-based techniques.(As shown in Figure 1, image from (Karnalim et al. [2019]))

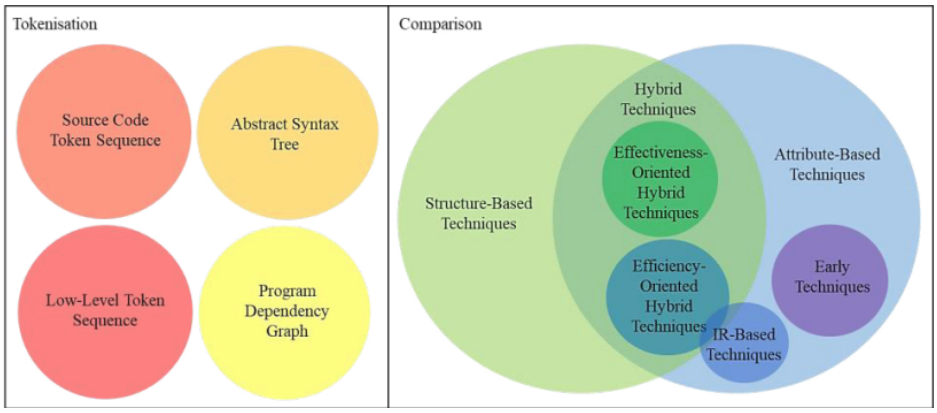

Figure 1: Tokenisation and Comparison

Plagiarized code can be separated into various levels defined by(Faidhi and Robinson [1987]). Figure 2, shows these plagiarism approaches in increasing order of sophistication and gives examples to describe them(Image from Karnalim et al. [2019]). Level 0 is easiest to detect, which is simply a verbatim copy of the original, the most challenging is level 6 which is a logic change, and is only considered as a plagiarism evidence if it occurs in combination with other attack levels.

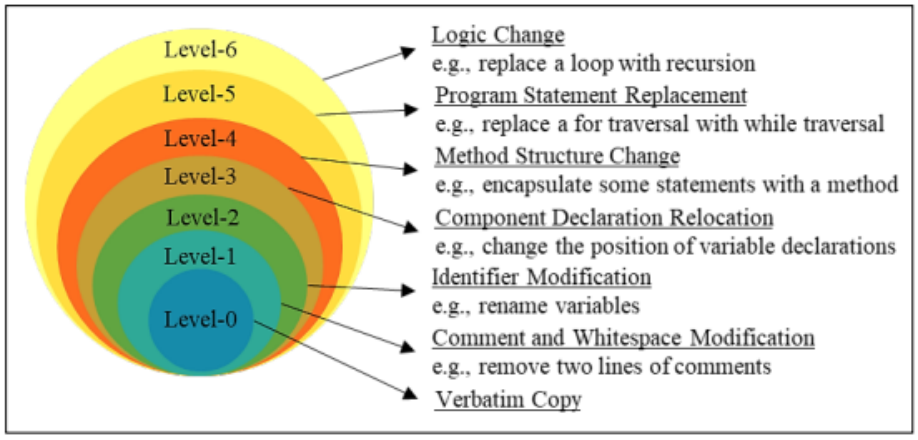

Figure 2: Plagiarism Approaches in Increasing Order of Sophistication

3.2 Challenges with AI generated code

This project hopes to address the challenge of detecting AI-generated code. The rise of AI-generated code presents a unique challenge for traditional plagiarism detection techniques. Large language models like Chat GPT, can generate code snippets that may be semantically similar to human-written code, but with different syntax, variable names or structures. These variations makes it difficult for conventional algorithms to detect plagiarism effectively as they typically focus on direct, surface-level similarities.

3.3 Evaluation Metrics

In order to evaluate the effectiveness of the plagiarism detection algorithm developed in this study, several evaluation metrics will be used. The metrics used will help to quantify the accuracy and overall performance of the detection technique. *Precision, Recall and F1* are the most commonly used metrics in plagiarism detection and classification evaluation. A trade off exists between recall and precision, and F1 score is used to balance these two metrics (Vujović et al. [2021]).

Precision measures the proportion of true positive plagiarism detections out of all instances that were flagged as plagiarism. It quantifies how accurate the algorithm is, high precision means that the algorithm is fairly accurate. It can be calculated using the formula:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \tag{1}$$

Where:

- **True Positives (TP)**: Correctly predicted positive cases.
- **False Positives (FP)**: Incorrectly predicted positive cases (negative cases that were incorrectly classified as positive).

Recall, however measures the proportion of actual instances of plagiarism that were correctly identified by the algorithm. High recall means that the algorithm is good at detecting most of the plagiarism. It is given by:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \tag{2}$$

Where:

- **True Positives (TP)**: Correctly predicted positive cases.
- **False Negatives (FN)**: Incorrectly predicted negative cases (positive cases that were incorrectly classified as negative).

A trade off exists between recall and precision, and **F1 score** is used to balance these two metrics, by offering a single value that combines them.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3}$$

The F1 score ranges from 0 to 1, where 1 indicates perfect precision and recall.

Scalability and robustness are also important factors to evaluate. A good algorithm should be able to handle a variety of coding languages. But for the sake of this project only Java code was used for testing, but considerations were made towards making a detection algorithm that is capable of handling various languages.

4 Methodology/Implementation

4.1 Choice of Algorithm

The algorithm used in this study is representative of a distinct approach to plagiarism detection. Due to its popularity and significance in education the MOSS (Measure of Software Similarity) algorithm was used, this is a tokenization and fingerprinting algorithm. The MOSS algorithm works by breaking down code into 'fingerprints' and comparing the frequency and arrangement of those fingerprints to detect similarities.

4.2 Design of Plagiarism Detection Algorithm

This study involved the development and custom implementation of the MOSS algorithm. The algorithm developed was tailored specifically to handle various programming languages by using the 'pygments' python library which offers multi-language support - this project focused exclusively on java code samples as the dataset consisted of only java samples.

Implementing an independent version of the MOSS algorithm provides several advantages over relying on the original Stanford-hosted system. First, it removes dependence on external servers, which eliminates issues related to downtime or internet connectivity. Second, it offers the ability to tweak the implementation. Lastly, it provides transparency and control, allowing the monitoring and modification of each step.

4.2.1 Description of Algorithm: MOSS

MOSS (Measure of Software Similarity) is a widely recognized plagiarism detection system designed to compare source code files and identify similarities. Developed by Alex Aiken in 1994, MOSS is effective in detecting exact matches and subtle changes in code. For this study the algorithm was developed in Python, using MOSS's winnowing method to detect code plagiarism(Schleimer et al. [2003]). The winnowing method works by generating tokens of the input code and generating k-grams, which are contiguous sequences of k tokens. A hash value is computed for each k-gram, and a fingerprint is created by selecting specific hashes based on the window size. These fingerprints are then compared to determine similarity. This approach is robust against techniques like renaming variables or changing the order of non-essential code blocks.

The algorithm follows several key steps:

1. **Normalization of Code:** The first step is to normalize the code. The code is normalized to ensure consistency in the subsequent steps. This includes actions like replacing variable names and function names with generic placeholders. The purpose of this is to help identify similarities even when elements like variable names are changed.

2. **Tokenization of Source Code:** The next step is to generate tokens from the normalized code. Tokenization is the process of breaking the normalized code into a sequence of meaningful units, such as keywords, operators and identifiers. The tokenization was achieved using the Python library 'Pygments' which offers lexers for various programming languages.

3. **Generation of k-grams:** The normalized tokens are grouped into overlapping sequences of k tokens, known as k-grams. These k-grams

capture the essence of the code and serve as the foundation for similarity detection.

4. **Hashing of k-grams:** Each k-gram is hashed using Python's hashlib function, which produces a numerical representation of the k-gram. The purpose of this hashing process is to reduce the dimensionality of the data while maintaining uniqueness across different k-grams.

5. **Windowing and Winnowing:** The hashed k-grams are grouped into fixed-size windows, and the winnowing method is applied to select representative hashes from each window. This is a crucial part of MOSS that helps to reduce the amount of data while preserving meaningful structure. The winnowing process identifies the minimum hash value in each window and records it with its position, this becomes part of the fingerprint. This minimizes redundancy and ensures that only the most distinctive features of the code are preserved for comparison.

6. **Fingerprint Generation:** The fingerprints generated through winnowing form the core of the similarity detection process. These fingerprints are used to compare different code files and calculate a similarity score, which reflects the amount of overlap between the two sets of fingerprints.

Winnowing with some sample text(Schleimer et al. [2003])

(a) Some text:

A do run run run, a do run run

(b) The text with irrelevant features removed:

adorunrunrunadorunrunrun

(c) The sequence of 5-grams derived from the text:

adoru dorun orunr runru unrun nrunr runru unrun nruna runad unado nador adoru dorun orunr runru unrun

(d) A hypothetical sequence of hashes of 5-grams:

77 74 42 17 98 50 17 98 8 88 67 39 77 74 42 17 98

(e) Windows of hashes of length 4: [1]

(77, 74, 42, 17) (74, 42, 17, 98) (42, 17, 98, 50) (17, 98, 50, 17) (98, 50, 17,98)(50, 17, 98, 8) (17, 98, 8, 88) (98, 8, 88, 67) (8, 88, 67, 39) (88, 67, 39, 77) (67, 39, 77, 74) (39, 77, 74, 42) (77, 74, 42, 17) (74, 42, 17, 98)

(f) Fingerprints selected by winnowing: [2]

17 17 8 39 17

(g) Fingerprints paired with 0-base positional information:
[17,3] [17,6] [8,8] [39,11] [17,15]

4.2.2 Tools and Programming Languages used

Key tools and libraries used:

- **Pygments:** For tokenizing code.
  Pygments is a powerful syntax highlighting library that supports a wide range of programming languages. In this project, it was used to tokenize source code in a language-aware manner, ensuring that only syntactically meaningful elements were retained. Its ability to distinguish between comments, whitespace, and actual code components made it especially valuable for accurate normalization and tokenization.

- **Hashlib:** For generating hash values of k-grams.
  Hashlib is a standard Python library that provides secure hash functions. It was employed in this project to generate unique hash values for each k-gram. Although the built-in Python 'hash()' function was used during early development for simplicity, Hashlib offers more consistency and reproducibility.

The development process began with a simplified version that operated on plain text strings, allowing for initial testing and debugging of the core winnowing logic.

[1] A *window* is a sliding group of consecutive hashes (in this case, 4 at a time).

[2] For each window, the minimum hash value is selected as the representative fingerprint. If a minimum hash has already been selected at that same position in a previous window, it is not repeated.

Once this version was functional, the algorithm was adjusted to handle code, incorporating the Pygments library to ensure accurate tokenization and normalization.

This iterative approach was facilitated thorough testing at each stage, resulting in a robust and flexible implementation suitable for handling both human-written and AI-generated code.

4.2.3 Comparator Functions for Similarity Scoring

The second step for plagiarism detection is the comparison stage. The fingerprints generated from different code submissions are compared, if enough fingerprints match between two files, the files are flagged as potentially plagiarized. Three different comparison functions were implemented in Python based on different approaches: Jaccard Similarity(Niwattanakul et al. [2013]), Cosine Similarity(Xia et al. [2015]), and Levenshtein Distance(Po [2020]). Each one operates on the fingerprints generated by the MOSS implementation.

For the main evaluation and testing in this project, Jaccard Similarity was selected as the primary comparator. This decision was based on its simplicity and effectiveness. A more detailed analysis of the comparator functions and the justification for this choice will be provided in a later section.

1. **Jaccard Similarity**

   Jaccard Similarity is a metric used to measure the similarity between two sets. It is defined as the size of the intersection between the two sets, divided by the size of the union of the two sets. Formally, for two sets A and B, the Jaccard Similarity is given by:

   $$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{4}$$

   Where:

   - $|A \cap B|$ is the number of elements common to both sets.
   - $|A \cup B|$ is the total number of unique elements across both sets.

   The higher the Jaccard Similarity, the greater the overlap between the two pieces of code. Jaccard Similarity is good at detecting literal copying or high overlap between code samples, order is not critical and it only checks the proportion of shared elements.

2. **Cosine Similarity**

   Cosine Similarity measures the cosine of the angle between two non-zero vectors in a multi-dimensional space. For two vectors $\mathbf{A}$ and $\mathbf{B}$, it is defined as:

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} \tag{5}$$

   Where:

   - $\mathbf{A} \cdot \mathbf{B}$ is the dot product of the vectors.
   - $\|\mathbf{A}\|$ and $\|\mathbf{B}\|$ are the magnitudes (norms) of the vectors.

   The result ranges from 0 to 100, where 100 indicates identical vectors (maximum similarity), and 0 no similarity. In this method, the fingerprints of each code sample are represented as frequency vectors. The dot product and magnitudes of these vectors are computed to calculate the Cosine Similarity. This captures the presence and the frequency of specific fingerprints, making it suitable for cases where certain patterns occur repeatedly. Cosine Similarity is good at detecting structural or logical similarities in code elements even if the format is altered.

3. **Levenshtein Similarity**

   Levenshtein Similarity is based on the Levenshtein distance, which measures the minimum number of single-character edits required to transform one string into another. The similarity is computed as:

$$\text{Levenshtein Similarity} = 1 - \frac{D}{\max(L_1, L_2)} \tag{6}$$

   Where:

   - D is the Levenshtein distance between the two strings.
   - L_1 and L_2 are the lengths of the two strings.

   The similarity score ranges from 0 to 100, where 100 indicates identical strings, and 0 indicates completely different strings. This comparator converts the fingerprints into strings and calculates the Levenshtein distance between them. The result is normalized to provide a similarity score. This method is effective in detecting small variations in code, such as reordering or slight modifications of statements.

4.3 Implementation of Plagiarism Detection Algorithm

4.3.1 Parameter Tuning and Optimization

In order to enhance the accuracy and reliability of the MOSS algorithm developed, a process of parameter tuning and optimization was undertaken. The algorithms sensitivity to differences in various source code samples is dependent on two key parameters: the *length of the k-grams* and the *window-size* used in the winnowing process (Schleimer et al. [2003]). To determine the optimal values for these parameters, a systemic approach was deployed.

The choice of k for k-gram fingerprinting can significantly affect plagiarism detection, a small value for k may flag common code structures as plagiarism, while a large k may overlook similarities. To determine the best value for k, parameter tuning was conducted using a dataset of non-plagiarized code samples(half of the samples to avoid leakage).
To evaluate k values:

1. Normalize and tokenize the code samples using the MOSS class.
2. Generate k-grams for each code sample using a range of values for k (2-50).
3. Compute the similarity between all pairs, by counting the number of common k-grams and dividing by the total number of k-grams.
4. Count only the highly similar pairs for each k (flagged as plagiarized).
5. Plot the results, showing how the number of detected similarities changes as k changes.

The experiments revealed that small numbers for k resulted in many unrelated code samples being flagged as similar, as k increased the number of flagged similarities decreased, indicating a reduction in false positives. However, beyond a certain value for k the number of detected similarities dropped sharply, and legitimate cases of plagiarism were being missed.

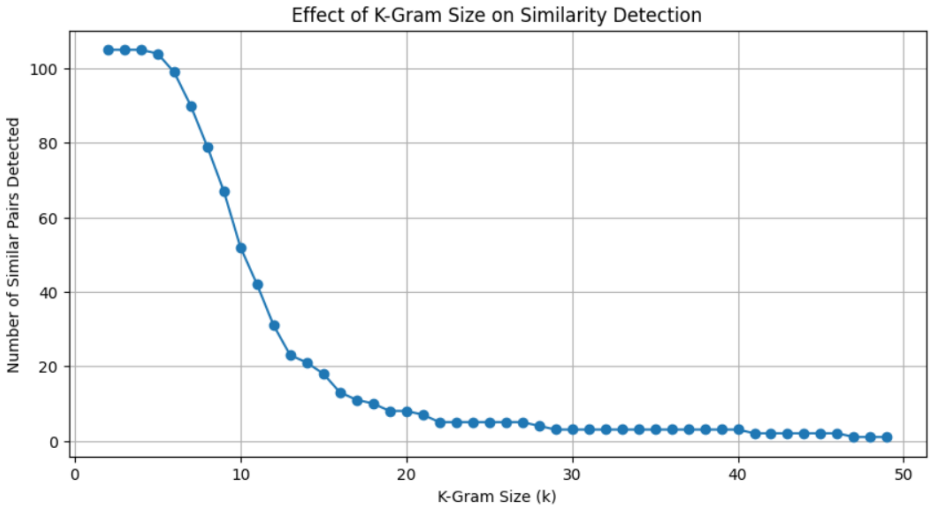

Figure 3: K-gram Graph

The optimal value for k was found to be at the point after the sharp decline, which is determined to be 15. It was also considered that the optimal values when working with different coding languages may vary, but for the case of this project only java code was considered when performing the parameter tuning as only java code was included in the dataset.

The window size w in the winnowing algorithm plays a crucial role in determining which fingerprints are selected as plagiarized. A small w results in more fingerprints being flagged as plagiarized, it flags common and non-significant code patterns, but a very large value for w can potentially miss actual cases of plagiarized code. The optimal value for w is where the similarity scores for plagiarized code is high and the similarity scores for non-plagiarized code is lower. To find the optimal value for w, we conducted parameter tuning by comparing similarity scores of plagiarized code samples and non-plagiarized code samples against an original code source. The goal was to find a w that maximized the difference between these similarity scores, this would ensure that plagiarized and non-plagiarized code would be distinguishable by the algorithm. Parameter tuning was conducted using a dataset of non-plagiarized code samples, plagiarized code samples(only half, to avoid leakage) and an original code sample.
To evaluate window sizes:

1. Define a range of window sizes to test, ensuring it covered a broad range of values.

2. Normalize and tokenize the code samples using the MOSS class.

3. Similarity computation, for each value of w we computed pairwise the similarity scores.

4. Graph the results to visualize the trends and support the selection of an optimal value.

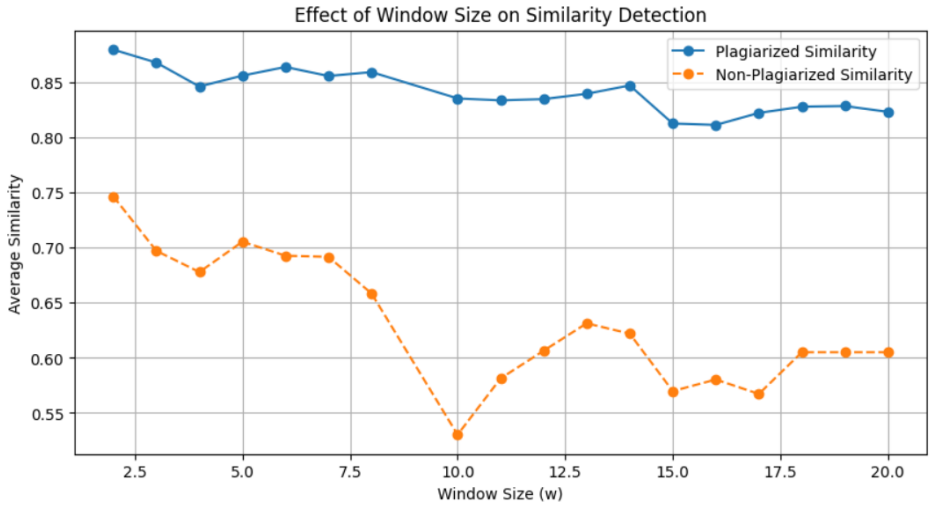

Figure 4: Window Size Graph

The optimal value for *window size* was more difficult to determine as no one value outperformed others greatly for the set of values that were tested, in this particular graph the value 10 does perform very well, and this is the case across all other tests (any value above 10 performs well in distinguishing plagiarized code). The value chosen was 10.

4.3.2 System Architecture

The system architecture of this project is structured into two main components: **MOSS-based fingerprint generation** and **Similarity comparison**. By separating the fingerprint generation from the similarity compari-

son, the system ensures modularity, allowing for flexibility in adjusting detection/comparison methods. The **MOSS class** serves as the core of the process and generates unique fingerprints for code input. Once fingerprints are generated, the **Comparator class** analyses the similarity between two fingerprints and produces a similarity score.

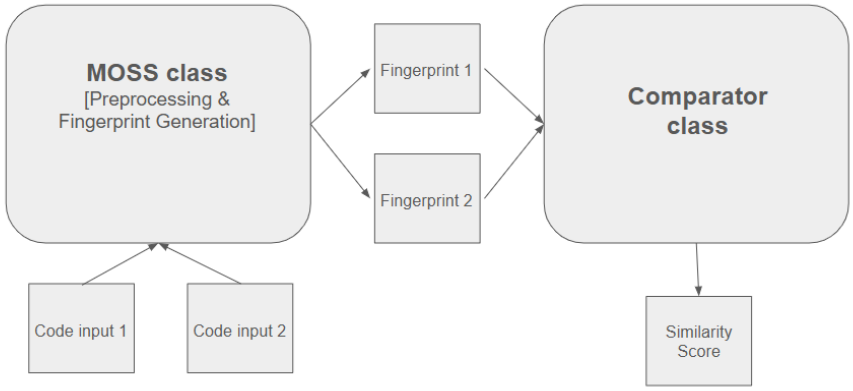

Figure 5: System Architecture

4.4 Dataset Preparation

Datasets of human-written code and a dataset of AI-generated code were used for this research.

The human-written source code plagiarism dataset retrieved from Karnalim [2019](Source Code Plagiarism Dataset). The dataset consists of 467 Java source code files covering seven different programming tasks. Each task contains three directories. 'Original' contains the original code task, 'non-plagiarized' contains N-subdirectories each representing one code file, 'plagiarized' contains six sub-directories representing the plagiarism levels from Faidhi and Robinson [1987]. The non-plagiarized code samples were generated by individual volunteers each tackling the task independently, while the plagiarized code samples were generated by the same group but explicitly applying one of the plagiarism strategies. The details of the dataset can be seen in the corresponding paper(Karnalim et al. [2019]).

The seven tasks the dataset contains:

- T1 (Output): Write a program that prints "Welcome to Java" five times.

- T2 (Input): Write a program that accepts the radius & length of a cylinder and prints the area & volume of that cylinder. All inputs and outputs are real numbers. Cylinder's area = radius * radius * 3.14159, Cylinder's volume = area * length.

- T3 (Branching): Write a program that accepts the weight (as a real number representing pound) and height (as two real numbers representing feet and inches respectively) of a person. Upon accepting input, the program will show that person's BMI (real number) and a piece of information stating whether the BMI is categorized as underweight, normal, overweight, or obese. A person is underweight if $BMI < 18.5$; normal if $18.5 \leq BMI < 25$; overweight if $25 \leq BMI < 35$; or obese if $BMI \geq 35$. The height in inches is calculated as Height = Feet $\times$ 12 + Inches, and BMI is given by:

$$BMI = \frac{\text{Weight} \times 0.45359237}{(\text{Height} \times 0.0254)^2}$$

- T4 (Looping): Write a program that shows a conversion table from miles to kilometers where one mile is equivalent to 1.609 kilometers. The table should display the first ten positive numbers as miles and pair them with their respective kilometer representation.

- T5 (Method): Write a program that accepts an integer and displays that integer with its digits shown in reverse. You should create and use a method void reverse(int number) which will show the reversed-digit form of the parameterized number.

- T6 (Array): Write a program that accepts 10 integers and shows them in reversed order.

- T7 (Matrix): Write a program that accepts a 4 x 4 matrix of real numbers and prints the total of all numbers placed on the leading diagonal of the matrix. You should create and use a method double sumMajorDiagonal(double[ ][ ] m) which will return the total of all numbers placed on the leading diagonal of the parameterized matrix.

The AI-generated code dataset consists of seven directories for each of the programming tasks, each has two subdirectories 'Naive' and 'Adaptations'. The 'Naive' directory, was populated using ChatGPT, where the tasks were given directly as prompts and the solutions were used to create code samples. The 'Adaptations' dataset was populated by giving ChatGPT its previous solutions and prompting it to make adaptations by using different approaches for each code sample, this was done to introduce variation.

4.5 Evaluation

After conducting parameter optimization, the MOSS-based plagiarism detection algorithm was evaluated using *precision, recall and F1 scores* to assess its effectiveness. The evaluation was conducted using the Source Code Plagiarism Dataset (Karnalim [2019]), to reduce leakage, when conducting parameter tuning only half of the dataset was used. Data leakage occurs when information from the test set is used during the model development or tuning, leading to overly optimistic performance estimates. To evaluate the algorithm, the entire dataset was used, the algorithm was applied to the code samples to detect similarities and classified the samples as plagiarized or not plagiarized.

Evaluation	**Precision**	**Recall**	**F1 Score**
Task 1	0.89	0.62	0.73
Task 2	0.96	1.00	0.98
Task 3	0.95	1.00	0.98
Task 4	0.96	0.82	0.89
Task 5	0.95	0.91	0.93
Task 6	0.95	0.99	0.97
Task 7	0.95	1.00	0.97

Table 1: Evaluation Results of MOSS Algorithm

The **scikit-learn** Python package was used to compute precision, recall and F1 scores, and evaluate the algorithm's accuracy. The results across the evaluations for the seven tasks demonstrated high precision, indicating that the algorithm effectively minimized false positives. The recall values varied slightly, some evaluations showing perfect recall, meaning that all plagiarized samples were detected, while others had minor false negatives. The F1

scores, which balance precision and recall, ranged from 0.73 to 0.98, showing strong overall detection accuracy. These results highlight the accuracy of the algorithm in identifying plagiarism. Table 1 shows the results. The values shown in the table represent the performance of the MOSS algorithm across the seven tasks, the values are expressed as decimals between 0 and 1, where values closer to 1 indicate better performance. For example, a precision value of 0.89 means that 89 percent of files flagged as plagiarized by the algorithm were actually plagiarized.

5 Evaluation: Results and Analysis

Detecting AI-generated code presents a challenge, as a single, standalone piece of AI-generated code may not be distinguishable from human-written code when simply using similarity detection. Simply comparing two code samples does not reveal whether or not a sample is generated by an AI model. However, this analysis operates on the hypothesis that if AI-generated code exhibits consistent patterns and similarities across multiple instances, it may be identifiable through large-scale comparisons.

To investigate this further, a series of experiments were conducted to uncover any patterns within AI-generated code and see whether it exhibited high similarity and then compare it to non-plagiarized human-written code. This study examines whether AI-generated code follows identifiable patterns and the following sections present the results of these experiments.

5.1 Investigating Similarities in AI-generated code

A self-similarity test was conducted on the AI-generated code samples. The goal was to determine whether AI-generated code samples exhibit any similarities that could influence AI detection.

1. **The 'Naive' Dataset Test:** This dataset consists of code samples generated by directly prompting ChatGPT to solve a programming task. The results over the seven programming tasks revealed relatively high similarity between the samples, with some cases showing near-identical outputs, like the cluster seen in the bottom left of Figure 6. This suggests that the AI model generates highly deterministic responses. Figure 6 uses a heatmap to visualize pairwise similarity

scores, it highlights the clusters of the highly similar code samples in the 'Naive' dataset(for one of the tasks). It is also noted that the overall average similarity for AI to AI code samples is much higher than a comparison with non-plagiarized samples(usually averages around 40 percent similarity or less, with few samples having high similarity).

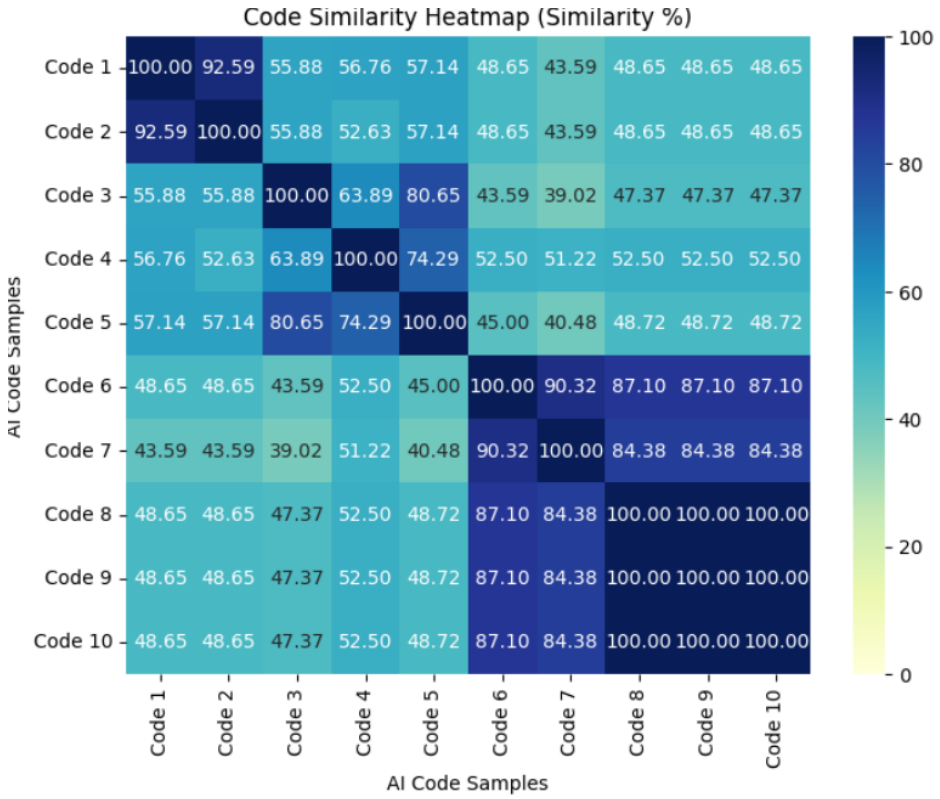

Figure 6: Self-Similarity Heatmap

2. **The 'Adaptations' Dataset:** This dataset was generated by explicitly prompting ChatGPT to solve the same tasks using different approaches. The similarity results were lower compared to the 'Naive' dataset, this indicates that modifications in the prompt can lead to more diverse outputs, which have less similarities. Figure 7 uses a heatmap to visualize pairwise similarity scores, it highlights the scattered similarity distribution.

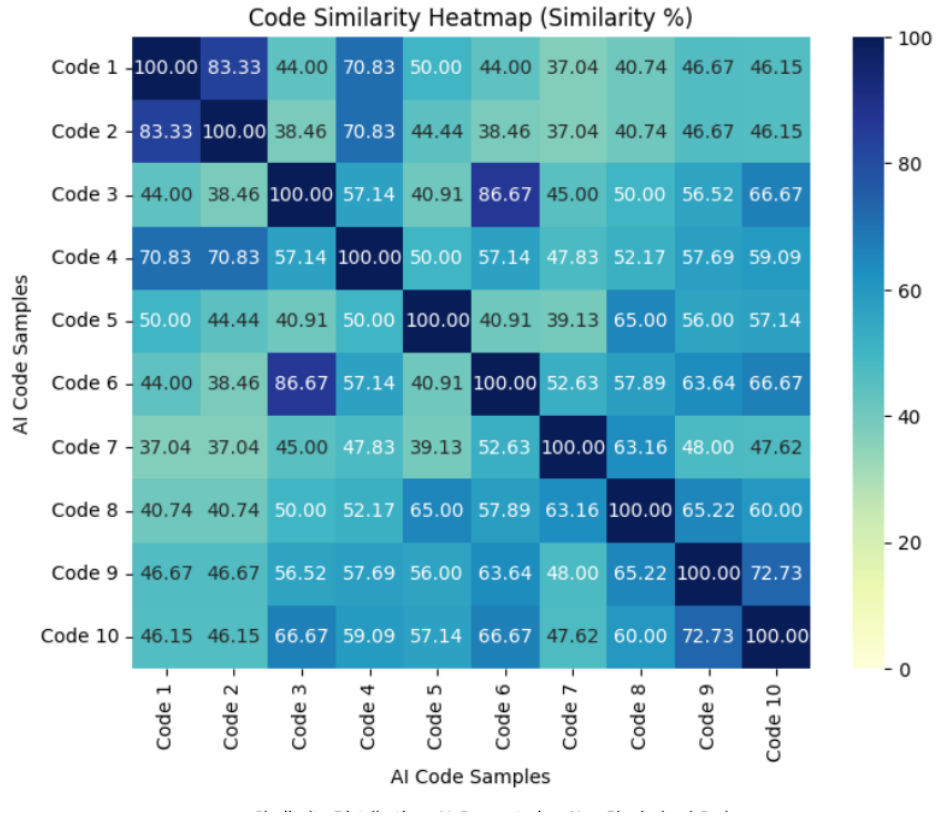

Figure 7: Self-Similarity Heatmap

5.1.1 Interesting Cases

1. **Identical AI-generated outputs:** One interesting observation was that, for certain programming tasks, the AI-generated solutions in the 'Naive' dataset were identical. In the case of this task: *Write a program that accepts 10 integers and shows them in reversed order*, many of the code samples generated were identical, as seen in Figure 8. This could be due to the task being part of the training data, as it is a very common introductory programming task and the AI may be generating responses that align closely with the learned patterns.

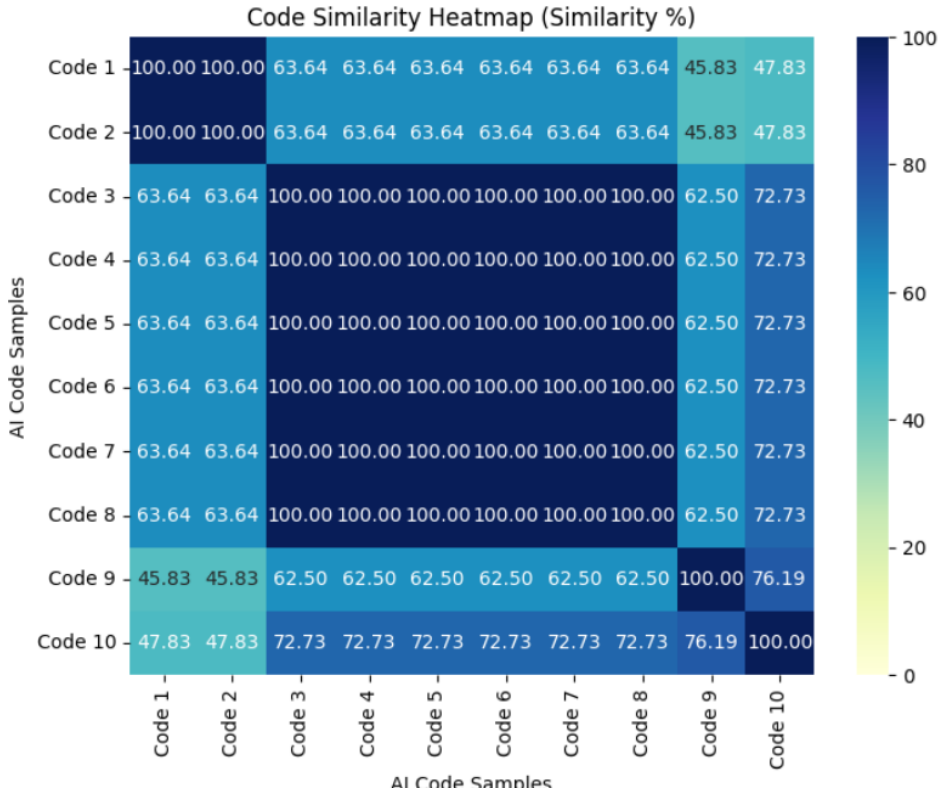

Figure 8: Interesting case of naively generated code samples with identical outputs

2. **AI Naive Output Clustered Similarity:** Based on the high similarity and clustering seen in the 'Naive' dataset, an experiment was run, where a larger quantity of outputs were generated for a particular task. These outputs were checked for similarity, and there were clusters of similar solutions. As seen in Figure 9, it highlights using different colours the clusters of highly similar code samples.

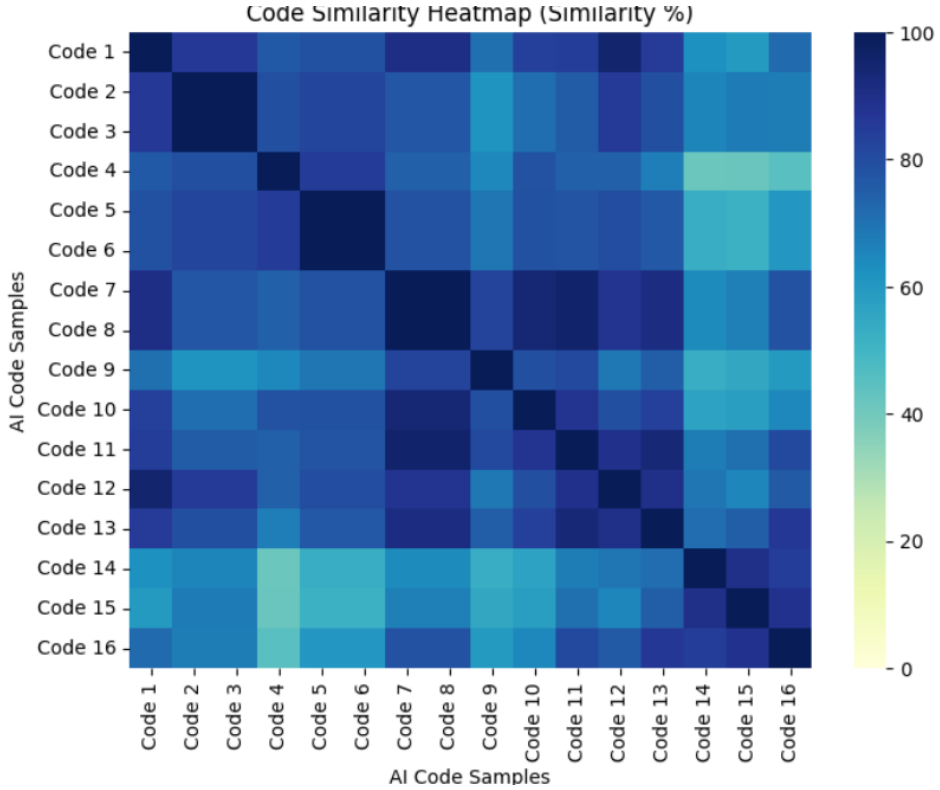

Figure 9: Interesting case of naively generated code samples with clusters

5.2 Testing Between AI-generated and Non-Plagiarized Code

This experiment aimed to determine whether AI-generated code exhibits any similarities to non-plagiarized human-written code for the same programming tasks. If AI-generated solutions closely resembles original human-written code, traditional plagiarism detection methods may fail to distinguish them effectively.

1. **The 'Naive' Code vs. Non-plagiarized:** Showed lower similarity scores overall (Figure 10), suggesting that AI-generated solution often follow distinct patterns that are less common in human-written code, which could help to distinguish them from human-written code. Figure 10 shows the results as a histogram, where the x-axis shows similarity scores generated when samples were compared, and the y-axis shows

the frequency of samples(AI vs Non-plagiarized comparisons) with a certain similarity score.

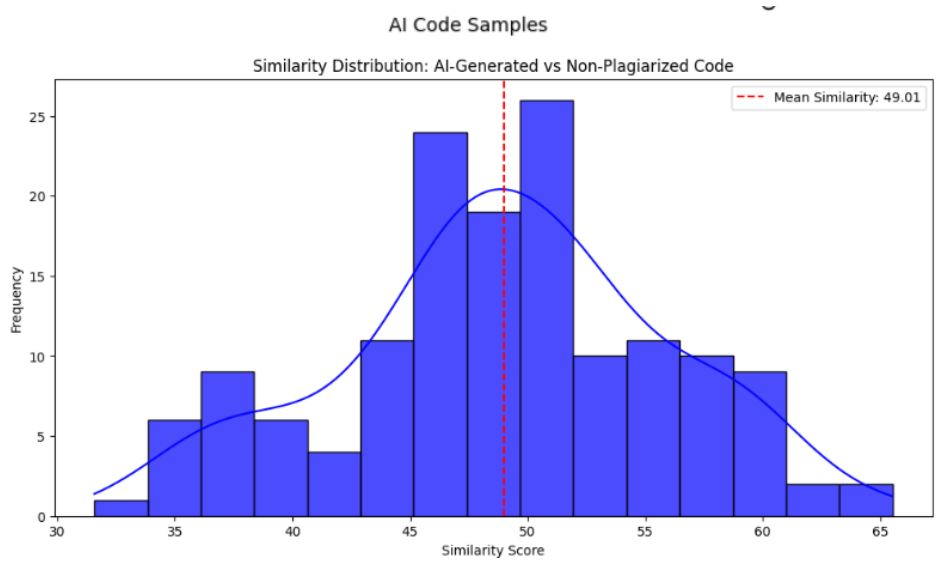

Figure 10: Naive VS Non-plagiarized

2. **The 'Adaptations' Code vs. Non-plagiarized:** Showed higher similarity with the human-written code samples than the 'Naive' dataset (Figure 11), indicating that modifying AI-generated code and using different approaches makes it more similar to human-written code and therefore harder to distinguish the two.

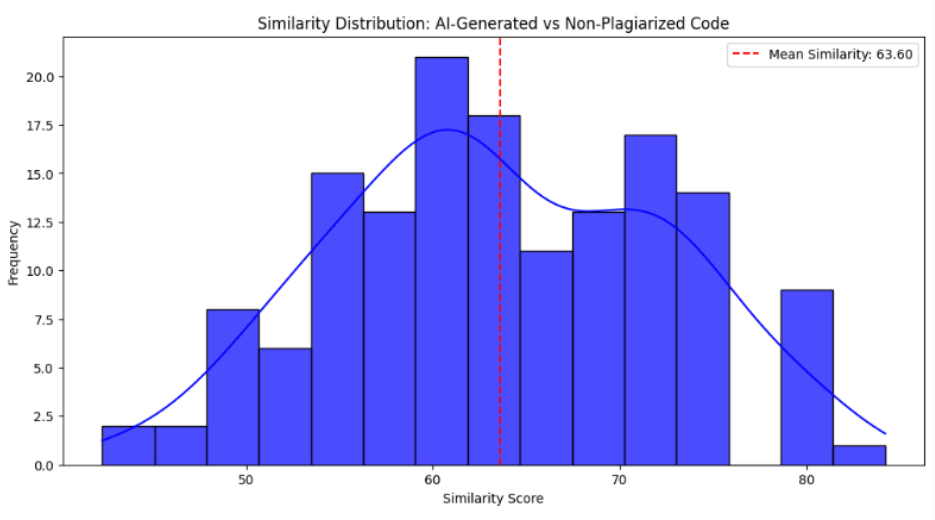

Figure 11: Adaptations VS Non-plagiarized

5.3 AI Model Output Similarity

Another experiment was conducted to check for similarities between various AI models. Different AI models were prompted to produce solutions to the same tasks: **ChatGPT**, **Claude** and **DeepSeek**. It's observed that different AI models often produce strikingly similar solutions when given the same programming task as a prompt directly with no modifications. The different models displayed high structural and syntactic similarities, many of the similarity scores ranged from 60-86 percent.

This could be due to models training on large-scale public datasets, if multiple models are trained on similar datasets, they may learn to generate similar solutions, especially for common programming tasks, examples of the outputs can be seen in Figures' 12, 13 and 14.

```java
import java.util.Scanner;

public class CylinderAreaVolumeCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Prompt user for radius and length of the cylinder
        System.out.print("Enter the radius of the cylinder: ");
        double radius = scanner.nextDouble();

        System.out.print("Enter the length of the cylinder: ");
        double length = scanner.nextDouble();

        // Calculate area and volume
        double area = radius * radius * 3.14159;
        double volume = area * length;

        // Output the results
        System.out.printf("The area of the cylinder is: %.4f\n", area);
        System.out.printf("The volume of the cylinder is: %.4f\n", volume);

        scanner.close();
    }
}
```

Figure 12: ChatGPT solution

```java
import java.util.Scanner;

public class CylinderCalculator {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Prompt user to enter radius and length
        System.out.print("Enter the radius of the cylinder: ");
        double radius = input.nextDouble();

        System.out.print("Enter the length of the cylinder: ");
        double length = input.nextDouble();

        // Calculate the area and volume
        final double PI = 3.14159;
        double area = radius * radius * PI;
        double volume = area * length;

        // Display results
        System.out.println("The area of the cylinder is " + area);
        System.out.println("The volume of the cylinder is " + volume);

        input.close();
    }
}
```

Figure 13: Claude solution

```java
import java.util.Scanner;

public class CylinderCalculator {
    public static void main(String[] args) {
        // Create a Scanner object to read input from the user
        Scanner scanner = new Scanner(System.in);

        // Prompt the user to enter the radius of the cylinder
        System.out.print("Enter the radius of the cylinder: ");
        double radius = scanner.nextDouble();

        // Prompt the user to enter the length of the cylinder
        System.out.print("Enter the length of the cylinder: ");
        double length = scanner.nextDouble();

        // Calculate the area of the cylinder
        double area = radius * radius * 3.14159;

        // Calculate the volume of the cylinder
        double volume = area * length;

        // Print the results
        System.out.printf("The area of the cylinder is: %.2f%n", area);
        System.out.printf("The volume of the cylinder is: %.2f%n", volume);

        // Close the scanner
        scanner.close();
    }
}
```

Figure 14: DeepSeek solution

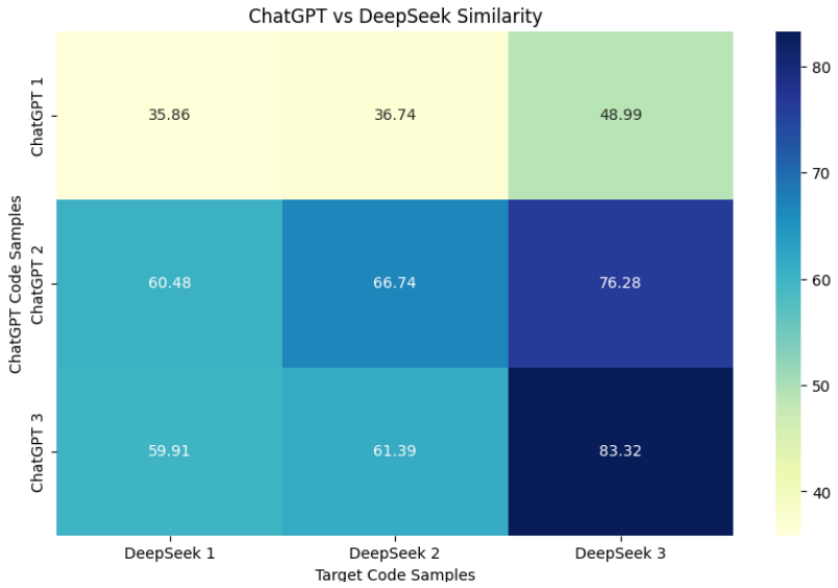

Figure 15: Similarity heatmap of ChatGPT vs Deepseek

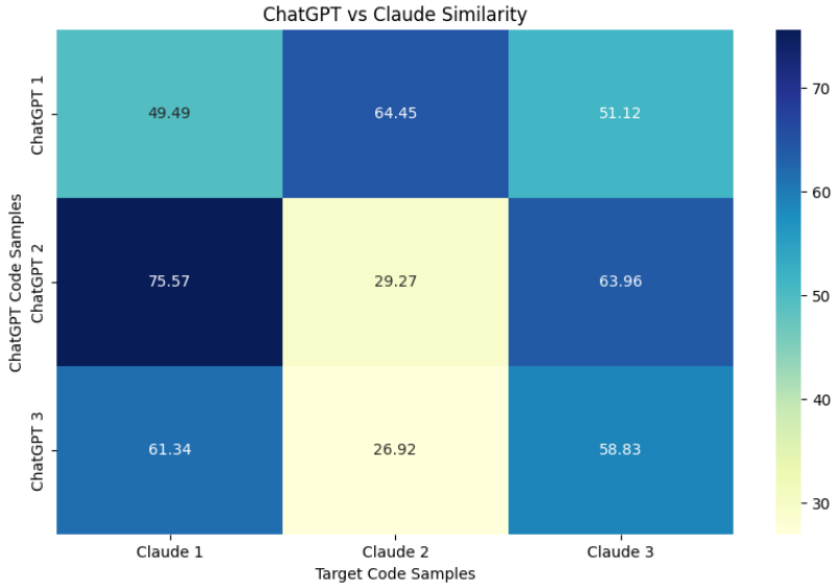

Figure 16: Similarity heatmap of ChatGPT vs Claude

5.4 Human Assessment of AI similarity

In some cases upon manual inspection the human eye can recognize structural and logical similarities between code samples, that the MOSS algorithm sometimes overlooks. For example, a 'for loop' and a 'while loop' are functionally equivalent, but if the syntax differs significantly, the MOSS algorithm may not see the similarity. This indicates there are some code alterations that can appear as 'different' to the algorithm, though a human observer would recognize the underlying similarity.The detection algorithm fails to recognize these similarities as it focuses on syntactic comparison.

Some examples of code modifications that the algorithm may miss include:

- Changing loop structures (for, while, do-while loops)
- Changing conditional statements
- Major reordering of functions and variable declarations
- Using different methods to achieve the same task (splitting and merging)

This limitation suggests that traditional token-based similarity detection might not always be sufficient for identifying plagiarism and AI-generated code, especially with the use of adaptations and different approaches to solving tasks.

5.5 Difference in Comparator Functions

To assess code similarity, three different comparator functions were implemented: **Jaccard Similarity**, **Cosine Similarity**, and **Levenshtein Similarity**. Each of these methods approach similarity measurement differently: Jaccard focuses on common tokens, Cosine evaluates vectorized representations of code, and Levenshtein considers edit distance. Despite these differences in methodology, the overall similarity scores produced by the three comparators were close, with only slight variations in their final results.

For this project, Jaccard Similarity was chosen as the primary comparison method due to several advantages. Firstly, it offers an intuitive and transparent way of measuring similarity, making it easier to interpret and reason about the results. Secondly, it allowed direct inspection of the shared fingerprints between code samples, which was particularly useful for debugging and analysis. Unlike Cosine Similarity, which requires converting the fingerprints into frequency-based vectors, or Levenshtein Similarity, which involves transforming them into strings for edit distance comparison, Jaccard operates directly on the original fingerprint sets without modifying them. This simplicity, combined with its effectiveness, made Jaccard a suitable and practical choice.

While each method provides a unique perspective on code similarity, they largely capture the same underlying patterns in the dataset. In most cases, the comparators agreed on which code samples were highly similar and which were not. This can be seen in Figures 17, 18 and 19.

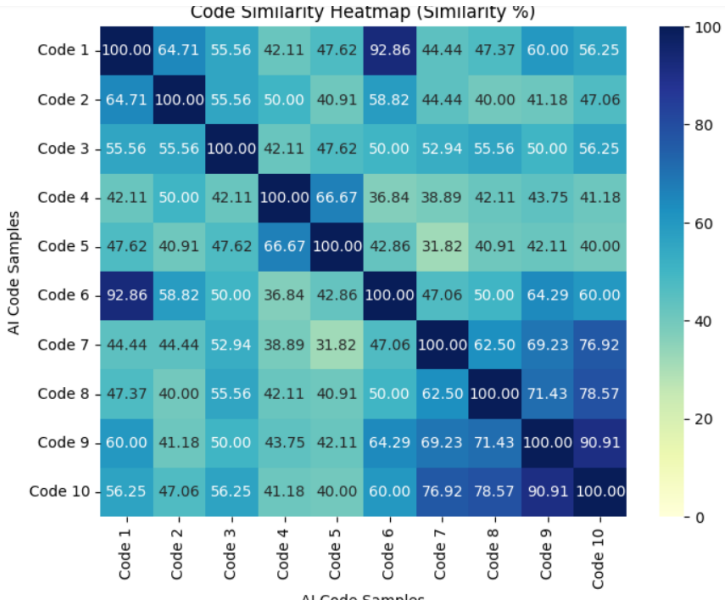

Figure 17: Jaccard Similarity Comparator

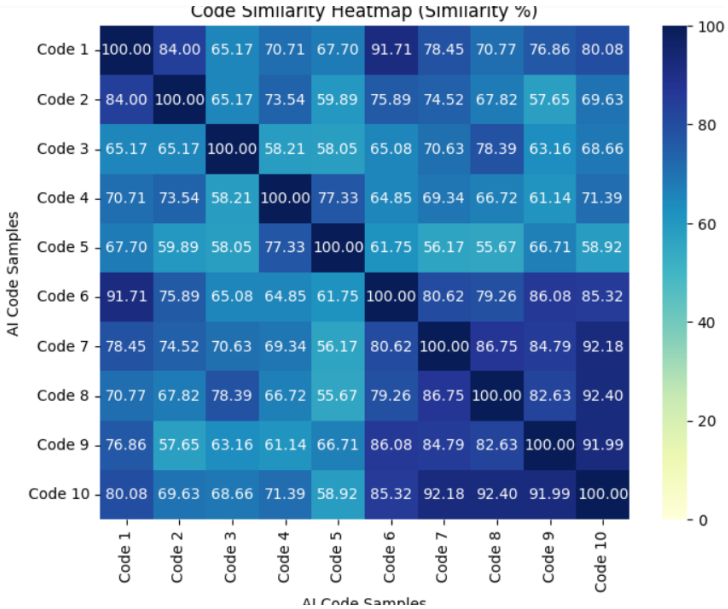

Figure 18: Cosine Similarity Comparator

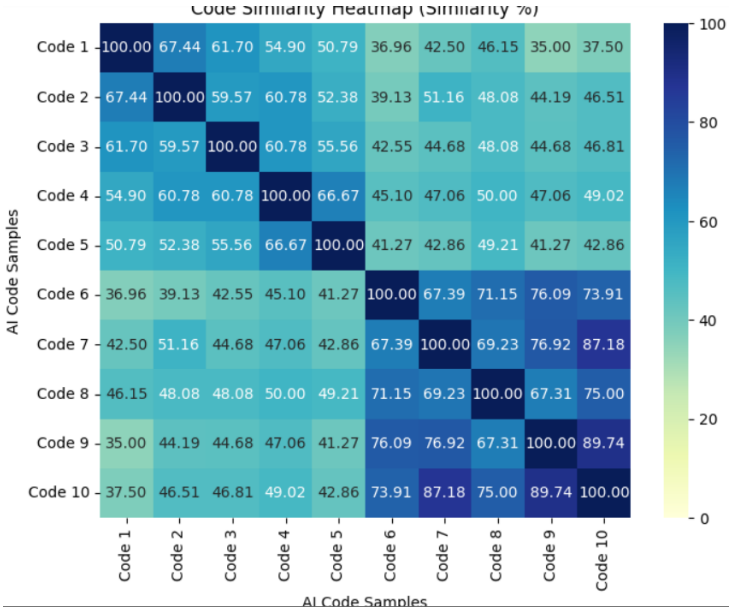

Figure 19: Levenshtein Similarity Comparator

6 Conclusions

6.1 Summary of Findings

The results of the AI self-similarity experiments indicate that AI-generated code samples, particularly in the **Naive** dataset, tend to exhibit a high degree of similarity. This suggests that AI models produce consistent patterns at times. This could be due to inherent biases in the model's code generation process.

A particularly notable finding was the presence of *identical* code samples within the **Naive dataset** for certain tasks, reinforcing the possibility that certain tasks may exist in the model's training set, leading to highly similar outputs. Additionally, the presence of *clusters* of similar solutions when generating multiple samples suggests that AI models may have a set of preferred solutions for specific problems. This clustering effect highlights the potential for detecting AI-generated code through large-scale similarity analysis.

These findings using the **Naive dataset** support the hypothesis that AI-generated code exhibits detectable patterns when analyzed collectively, which could be leveraged in AI-detection strategies.

The **Adaptations**, which was designed to introduce variation, resulted in a more dispersed similarity distribution, indicating that modifying the prompt or approach can reduce direct similarities between the AI-generated code samples, making it more difficult to identify AI-generated code. This suggests that existing plagiarism detection methods may need refinement to handle AI-assisted code generation effectively.

6.2 Scope and Limitations

This project focuses on the detection of plagiarism in AI-generated and human-written code. The scope includes the design and implementation of a distinct algorithm based on MOSS using token-based analysis. Datasets comprising both *human-written* and *AI-generated code* were used to test and evaluate the algorithm. The algorithm developed was evaluated using performance metrics like precision, recall, and F1 score.

However, the study has several limitations. First, the dataset used for evaluation may not fully represent the wide range of coding styles and complexities found in real-world applications, potentially affecting the generalization of the findings.

Second, while the algorithm is tested for performance and accuracy, this research does not include a detailed cost-benefit analysis of their computational efficiency in large-scale application.

Lastly, the study emphasizes the technical aspects of detection and does not address the ethical or policy implications of using AI-generated code in educational or professional settings.

6.3 Implications of Findings and Further Improvements

The findings indicate that while existing techniques like MOSS can effectively detect direct plagiarism, AI-generated code requires more nuanced strategies. One of the deficiencies of a token-based algorithm like MOSS is that it struggles in identifying deep structural similarities, it fails to handle logically equivalent but syntactically different code samples. This limitation indicates that more advanced techniques such as **Abstract Syntax Tree (AST) analysis** or **Graph based representations of the program logic** may be necessary to capture deeper structural similarities between code samples.

References

J. Faidhi and S. Robinson. An empirical approach for detecting program similarity and plagiarism within a university programming environment. *Computers & Education*, 11(1):11–19, 1987. ISSN 0360-1315. doi: https://doi.org/10.1016/0360-1315(87)90042-X. URL `https://www.sciencedirect.com/science/article/pii/036013158790042X`.

D. Fu, Y. Xu, H. Yu, and B. Yang. Wastk: A weighted abstract syntax tree kernel method for source code plagiarism detection. *Scientific Programming*, 2017(1):7809047, 2017. doi: https://doi.org/10.1155/2017/7809047.

O. Karnalim. Source code plagiarism dataset, 2019. URL `https://github.com/oscarkarnalim/sourcecodeplagiarismdataset`. GitHub repository, Accessed: 2024 - 2025.

O. Karnalim, S. Budi, H. Toba, and M. Joy. Source code plagiarism detection in academia with information retrieval: Dataset and the observation. *Informatics in Education*, 18(2):321–344, 2019.

C. Liu, C. Chen, J. Han, and P. S. Yu. Gplag: detection of software plagiarism by program dependence graph analysis. In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, page 872–881, 2006. doi: 10.1145/1150402.1150522.

S. Niwattanakul, J. Singthongchai, E. Naenudorn, and S. Wanapu. Using of jaccard coefficient for keywords similarity. In *Proceedings of the international multiconference of engineers and computer scientists*, pages 380–384, 2013.

M. Novak. Review of source-code plagiarism detection in academia. In *2016 39th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, pages 796–801, 2016. doi: 10.1109/MIPRO.2016.7522248.

D. K. Po. Similarity based information retrieval using levenshtein distance algorithm. *Int. J. Adv. Sci. Res. Eng*, 6(04):06–10, 2020.

S. Schleimer, D. S. Wilkerson, and A. Aiken. Winnowing: local algorithms for document fingerprinting. In *Proceedings of the 2003 ACM SIGMOD*

International Conference on Management of Data, page 76–85. Association for Computing Machinery, 2003. doi: 10.1145/872757.872770.

Ž. Vujović et al. Classification model evaluation metrics. *International Journal of Advanced Computer Science and Applications*, 12(6):599–606, 2021. doi: 10.14569/IJACSA.2021.0120670.

P. Xia, L. Zhang, and F. Li. Learning similarity with cosine similarity ensemble. *Information sciences*, 307:39–52, 2015.